

CH 8 DATABASE ARCHITECTURE

Authored by : Arpit Raj , Lnmiit jaipur

Database Architecture describes how users, application, business logic and databases communicate with each other in a DBMS.

1 Tier architecture

- user, application, database all exist on some machine, everything runs locally
- **Eg:** SQLite, Local MySQL, personal projects

2 Tier architecture

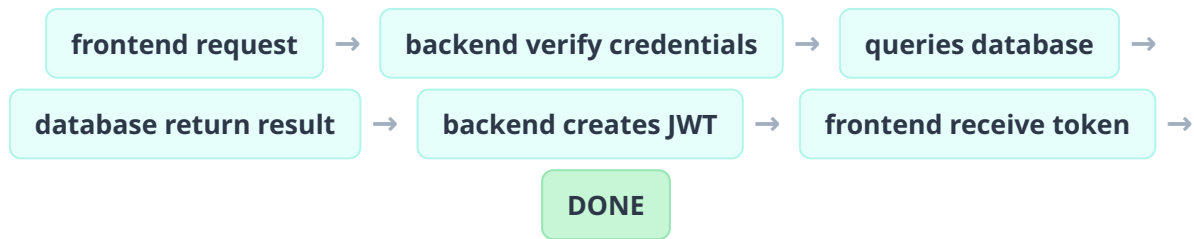
- consists of a client application and a database server
- **Eg:** Desktop banking application
- multiple users have a centralized database
- easy maintenance, however business logic is stored inside every client, if a bug found, need to update all computers
- **Eg:** C# desktop + SQL server
- legacy enterprise desktop software

3 Tier architecture

- **presentation layer** which handles the UI, HTML, CSS, react, android, IOS
- **Business logic layer** handles the authentication, authorization, validation, API's, business rules. this is basically our backend.
Eg: Express.JS / Springboot / Django / ASP.NET

- **Database layer** : stores the Tables, indexes, transaction, constraints

Eg: frontend request



PROS

- scalable
- secure
- centralized business logic
- easy maintenance
- better caching

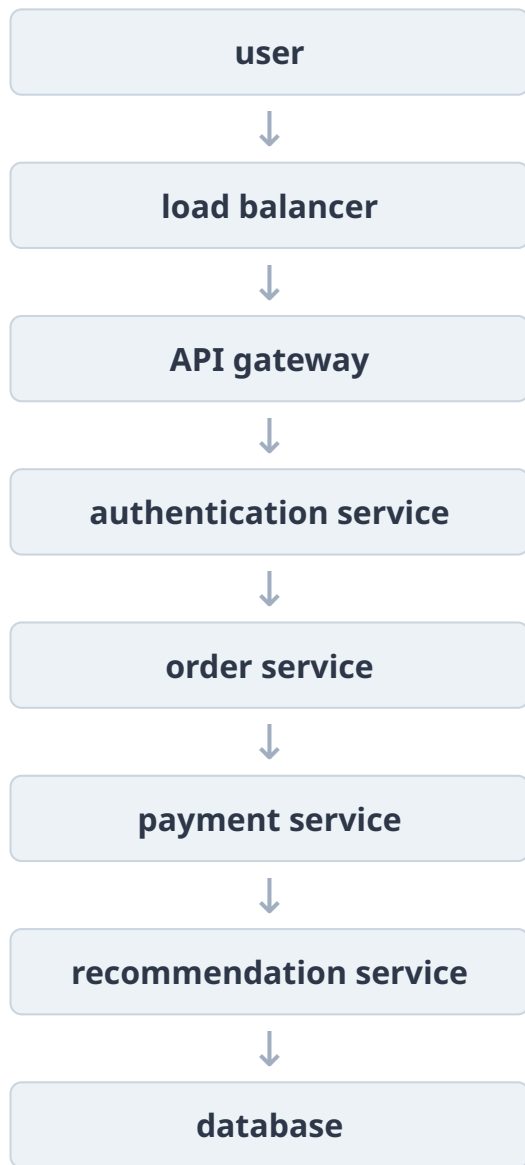
CONS

- more servers
- high cost
- complex
- additional network

N-Tier architecture

modern production systems instead of having one backend, they have many specialized services and each service often have their own database and backend.

so an N tier architecture introduces multiple intermediate services each with a specific responsibility.



hence :

- each service can scale independently
- fault isolation
- independent deployment
- better maintenance

↑ PROS

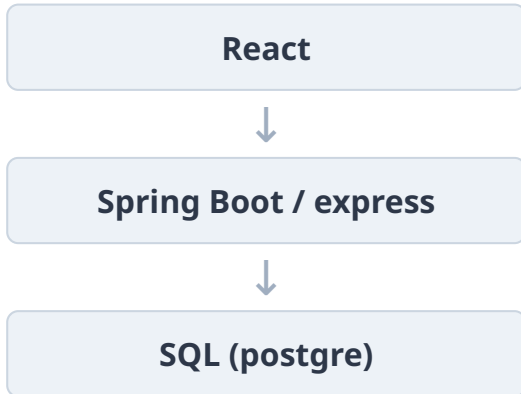
↓ CONS

- distributed transaction
- network latency
- complexity
- debugging

Industry Architectures

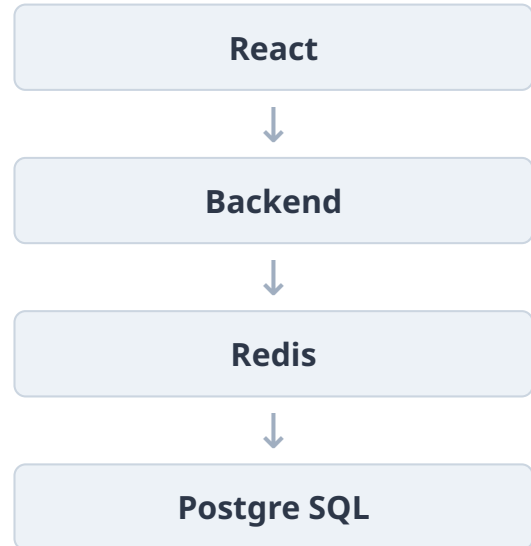
Startups

Simple 3 Tier



Medium companies

3 Tier with supporting infra



FAANG → N tier architecture

- we cant connect frontend directly to database because database credentials leaked, business logic bypass, security risks
- each microservice has own database such tht it enables indepent deployment, independent scaling, fault isolation.
- tradeoffs is complexity in maintaining consistency.

Practice Questions

Q1. Define database architecture. Why do different architectures exist?

A1. Database architecture describes how users, applications, business logic, and databases interact within a system. Different architectures exist because application requirements evolve with scale, security, maintainability, and performance needs.

Q2. Differentiate between 1-tier, 2-tier, 3-tier, and N-tier architectures.

A2.

1-tier: UI, application logic, and database on one machine.

2-tier: Client communicates directly with a remote database server.

3-tier: Client communicates with an application server, which interacts with the database.

N-tier: Multiple intermediate services (API gateways, authentication, microservices, caches, etc.) coordinate before accessing one or more databases.

Q3. Why is 3-tier architecture preferred for modern web applications?

A3. A 3-tier architecture separates presentation, business logic, and data storage. This improves security, maintainability, scalability, and allows business rules to be centralized on the application server rather than duplicated across clients.

Q4. Why should clients not communicate directly with the database in production systems?

A4. Direct database access from clients exposes credentials, bypasses business logic and validation, complicates authorization, increases attack surface, and places unnecessary load on the database. An application server mediates access and enforces business rules.

Q5. Explain the architecture of a Flipkart login request from the user's browser to the database.

A5.

1. The browser sends login credentials to the backend.
2. The backend validates the request and authenticates the user.
3. The backend queries the database for user information.
4. The database returns the result.
5. The backend generates a session or JWT.
6. The response is returned to the browser.

The browser never directly communicates with the database.

Q6. Why do large-scale systems often use N-tier architectures instead of a simple 3-tier model?

A6. Large systems use N-tier architectures because responsibilities can be divided into specialized services that scale independently, fail independently, and can be deployed separately. Although this introduces distributed systems challenges, it greatly improves scalability and maintainability for high-traffic applications.